CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY (CHARUSAT)

Mini-Project Report

MINIBANK

A Java Console-Based Banking Management Simulation

Submitted in partial fulfilment of the coursework for

CEUE203: OBJECT-ORIENTED PROGRAMMING

Submitted by:	Ka.Patel Dhyanam K.
Enrollment:	D26CE149
Subject:	CEUE203: Object-Oriented Programming

Academic Year: 2026–2027

Abstract

MiniBank is a Java console-based banking simulation developed to bring together object-oriented programming concepts in a single application. The application supports creating savings, current and fixed-deposit accounts; depositing, withdrawing and transferring whole-number amounts; viewing account statements; listing accounts; generating a summary report; and saving account data locally. It also demonstrates custom exceptions, runtime annotations and reflection, generic repository design, interfaces, inheritance, polymorphism, file input/output, and concurrent task execution. The supplied project explicitly states that it uses simulated local data and does not connect to a real bank or payment network.

This report describes the supplied implementation, its structure, principal workflows, validation rules, persistence approach, concurrency demonstration and available tests. Console output examples were captured by running the supplied Java application with a controlled sequence of inputs. They are illustrative test runs rather than evidence of connection to a real banking system.

Table of Contents

1. Introduction

2. Problem Statement

3. Objectives

4. Scope and Limitations

5. Tools and Requirements

6. System Design

7. Module and Class Description

8. Main Workflows and Algorithms

9. Object-Oriented Concepts Applied

10. Data Persistence and Concurrency

11. Testing and Validation

12. Sample Output Screenshots

13. How to Run the Project

14. Conclusion

15. References

1. Introduction

Banking operations provide a useful context for demonstrating object-oriented design because accounts have state, transactions change balances, and different account types follow different rules. MiniBank models these operations in a local Java application. A text menu accepts user choices and delegates operations to a service layer, while model classes represent accounts and their behaviour. A repository stores accounts in memory and can load or save them to a local text file.

The project is intended for learning and demonstration. Account values are represented as whole-number units using long, and the program does not use a real banking backend, payment gateway, or network service.

2. Problem Statement

The project addresses the need for a small, understandable application in which common account operations can be performed and observed from a console. The design should keep account-specific rules separate, reject invalid transactions, preserve account records between runs through a local data file, and demonstrate selected advanced Java topics such as generics, exceptions, annotations, reflection and concurrency.

3. Objectives

- Represent customer accounts using classes and inheritance.
- Support savings, current and fixed-deposit account types with different withdrawal rules.
- Provide account creation, deposit, withdrawal, transfer, statement, listing and summary-report operations.
- Validate user input and report banking errors with meaningful messages.
- Persist account information to a local file and reload it at startup.
- Demonstrate interfaces, records, enumerations, switch expressions, generics, Optional, annotations and reflection.
- Demonstrate concurrent tasks and synchronized balance operations.
- Provide a small set of automated service tests in the supplied source tree.

4. Scope and Limitations

4.1 Scope

- Console-based account management using a numbered menu.
- Savings, current and fixed-deposit account models.
- Deposit, withdrawal and account-to-account transfer.
- Account statement with interest-rate and estimated yearly-interest display.
- Account listing and aggregate report.
- Local file storage in data/accounts.txt.
- A batch/concurrency demonstration, anonymous-class and lambda withdrawal rules, resource closing and audit logging.

4.2 Limitations

- The README states that the application does not connect to a real bank or payment network.
- The program uses simulated local data only; it is not production banking software.
- Balances are stored as whole-number units (long), not decimal currency values.

- The console interface has no graphical user interface or network-based multi-user access.
- The supplied test suite contains three service tests; this report does not claim exhaustive testing.
- The archive includes a prebuilt JAR and compiled class folders, but the report is based primarily on the Java source, README and Maven configuration.

5. Tools and Requirements

Item	Requirement / use
Programming language	Java
Java version	JDK 17 or newer, as stated in README.md
IDE	IntelliJ IDEA can open the Maven project; the supplied README also describes running through VS Code or a terminal.
Build configuration	Maven pom.xml
Test framework	JUnit Jupiter 5.10.2 (test dependency in pom.xml)
Persistence	Local text file: data/accounts.txt
Application type	Console application

6. System Design

6.1 Architecture overview

The source follows a layered structure. The service.MiniBank class handles the menu and console interaction. service.BankService coordinates operations and calls the repository. repository.AccountRepository stores account objects and performs loading and persistence. model classes contain account data and rules. exception classes represent banking failures, while util classes provide annotation-based validation and audit helpers.

Layer / package	Responsibility
service	Console menu, banking workflows, transaction processor, concurrency and resource demonstrations.
model	Account abstraction, account subtypes, customer/address data, interfaces, enum and bank information record.
repository	Generic repository contract, in-memory account lookup and local file persistence.
exception	BankException hierarchy for invalid amounts, missing accounts and insufficient funds.
util	Custom annotations, validation helper and audit logging through reflection.
src/test/java	JUnit service tests for deposit, withdrawal rejection and concurrent deposits.

6.2 High-level workflow

1. Start application and attempt to load saved account data.
2. Display bank information and the numbered menu.
3. Read and validate the selected menu option.
4. Call the relevant service method for the requested operation.
5. Display the result or a handled banking/file error.
6. When the user selects Save and exit, write account data and terminate.

7. Module and Class Description

7.1 Main menu and service

service.MiniBank is the entry point and contains main(String[] args). It loads data/accounts.txt, displays the MiniBank information record, reads menu input, maps numeric choices to MenuOption values using a switch expression, and calls methods for opening accounts, deposits, withdrawals, transfers, statements, listing, reports, batch demonstrations and exit. service.BankService implements the main application operations and coordinates repository access.

7.2 Account model

Class / interface	Purpose in the supplied source
Account	Abstract base class holding account number, owner name, balance and active state; implements Transactable and InterestBearing.
SavingsAccount	Enforces a minimum balance and reports an interest rate of 4.0%.
CurrentAccount	Supports a configured overdraft limit and reports an interest rate of 0.0%.
FixedDepositAccount	Models a fixed-deposit account; the README states withdrawals are locked.
Transactable	Declares deposit and withdrawal operations.
InterestBearing	Declares interest-related behaviour implemented by account types.
WithdrawRule	Functional interface used for withdrawal-policy examples.
Customer / Customer.Address	Represents customer information and a nested address; Customer provides cloning logic.
BankInfo	Record used to display the bank name, hours and support email.
MenuOption	Enumeration of menu actions.
Premium	Marker interface for premium-service classification.

7.3 Repository and persistence

Repository<T> defines save, findById, findAll and deleteById. AccountRepository implements this generic contract using a ConcurrentHashMap keyed by account number. findById returns Optional<Account>, findAll returns a sorted list, and require throws AccountNotFoundException if the account does not exist. The repository reads and writes account records in data/accounts.txt. Owner names are Base64-encoded in the file format documented in the source code.

7.4 Exceptions and utilities

BankException is the checked base exception. InvalidAmountException, InsufficientFundsException and AccountNotFoundException specialize error cases. InsufficientFundsException also carries a shortfall value. The util package contains Audit, ValidateWith, MinValue and NotBlank annotations and a Validator helper that uses reflection to inspect annotations and log methods marked for auditing.

7.5 Transaction helpers

TransactionProcessor wraps a fixed thread pool and implements AutoCloseable so submitted tasks can finish before the resource is closed. TransactionBuffer provides a synchronized bounded queue using wait() and notifyAll(). ClosableDemo demonstrates try-with-resources and the close() method.

8. Main Workflows and Algorithms

8.1 Opening an account

7. Read account number, owner name, account type and opening balance.
8. For a savings account, read the minimum balance; for a current account, read the overdraft limit.
9. BankService validates required text, duplicate account numbers and non-negative values.
10. Create the selected account subtype and save it in the repository.
11. Display the newly opened account.

8.2 Deposit and withdrawal

Deposit and withdrawal first locate the account by its account number. Account.deposit rejects amounts less than or equal to zero and uses Math.addExact to detect long overflow. Account.withdraw rejects non-positive amounts and checks canWithdraw before changing the balance. The specific account subtype determines the withdrawal rule.

8.3 Transfer

A transfer requires distinct source and destination accounts and a positive amount. Account.transfer obtains locks in account-number order to reduce the risk of deadlock when transfers happen concurrently. It checks the source account withdrawal rule, subtracts the amount, and deposits it into the destination. If the destination deposit fails, the source balance is restored and a BankException is raised.

8.4 Statement and report

The statement displays the account representation, interest rate and estimated yearly interest. The report calculates account count, total balance, and counts of savings, current and fixed-deposit accounts based on the accounts currently held by the service.

8.5 Batch/concurrency demonstration

The menu option submits 1,000 one-unit deposits to a demo savings account using TransactionProcessor. The source prints the resulting balance and explicitly states that the demonstration account is not saved to bank data. It also checks withdrawal rules through an anonymous class and a lambda, demonstrates try-with-resources, and invokes audit logging.

9. Object-Oriented Concepts Applied

Concept	Evidence in the project
Encapsulation	Account state is stored in fields; public methods control balance operations.
Abstraction	Account is abstract and declares canWithdraw; interfaces define transaction and interest behaviour.
Inheritance	SavingsAccount, CurrentAccount and FixedDepositAccount extend Account.
Polymorphism	The service and report operate on Account references while account subtypes apply their own rules.
Interfaces	Transactable, InterestBearing, WithdrawRule, Repository<T> and marker interface Premium.
Records and enums	BankInfo record and MenuOption enum.
Generics and Optional	Repository<T> and Optional<T> lookup.
Exception handling	Checked banking exceptions, try/catch/finally and try-with-resources.

Annotations and reflection	Custom annotations are inspected by Validator.
Concurrency	ConcurrentHashMap, synchronized balance methods, ordered transfer locks and ExecutorService.
File handling	AccountRepository loads and persists account records.

10. Data Persistence and Concurrency

10.1 Data file format

The source documents the persisted record layout as accountNumber|base64(owner)|balance|type|rule. The type field distinguishes SAVINGS, CURRENT and FIXED accounts. The final field stores the minimum balance for savings, the overdraft limit for current accounts, or zero for fixed-deposit accounts. The application attempts to load this file when it starts and saves it when the user chooses option 9.

10.2 Thread safety

Account.getBalance, deposit and withdraw are synchronized. AccountRepository uses ConcurrentHashMap for concurrent map access. Transfers lock both involved accounts in a consistent account-number order. TransactionProcessor submits work to a fixed thread pool and waits for termination during close. These are educational synchronization mechanisms and do not make the application suitable for real financial use.

11. Testing and Validation

The supplied source includes BankServiceTest with three JUnit tests. The tests cover a deposit increasing a balance, a withdrawal beyond the allowed balance throwing InsufficientFundsException, and 1,000 concurrent deposits producing the expected balance. The archive includes compiled test classes, but Maven was not available in the report-generation environment, so the JUnit suite was not executed as part of preparing this report. The application source was compiled with javac and a scripted console run was performed to capture the output examples in the next section.

Test case in supplied source	Expected condition	Verification status for this report
depositIncreasesBalance	Balance 100 + deposit 50 becomes 150.	Source test present; not run through Maven here.
withdrawalBeyondAllowedBalanceThrows	Withdrawal of 200 from a savings balance of 100 with minimum 0 throws InsufficientFundsException.	Source test present; not run through Maven here.
concurrentDepositsKeepCorrectBalance	1,000 concurrent one-unit deposits result in balance 1,000.	Source test present; not run through Maven here.
Manual console run	Open account, deposit, transfer, statement, list, report, concurrency demo and save/exit.	Executed using the supplied Java source.

12. Sample Output Screenshots

The following monochrome console captures were generated from the actual output of the supplied application after compiling and running its Java source. The run used a scripted set of menu inputs and a copy of the supplied local data file. Existing sample data was loaded at startup; the screenshots show selected output sections rather than every menu repetition.

12.1 Account creation and deposit

```
MiniBank console output

Account opened: 456 | Sample Customer | Balance: 500 | Active: true
Deposit successful.
```

Figure 1. Opening a current account and depositing funds.

12.2 Transfer and account statement

```
MiniBank console output

Transfer successful.
123 | rr | Balance: 200 | Active: true
Interest rate: 4.00%
Estimated yearly interest: 8.00
```

Figure 2. Successful transfer and statement with interest estimate.

12.3 Account listing and report

```
MiniBank console output

123 | rr | Balance: 200 | Active: true
456 | Sample Customer | Balance: 650 | Active: true
========== BANK REPORT ==========
Account count: 2
Total balance: 850
Savings accounts: 1
Current accounts: 1
Fixed deposit accounts: 0
```

Figure 3. Listing accounts and generating account totals.

12.4 Concurrency and resource demonstration

```
MiniBank console output

Thread-pool demo: 1000 concurrent deposits completed. Final demo balance: 1000
This demonstration account is not saved to your bank data.
Withdraw rule (anonymous class): true
Withdraw rule (lambda): true
Using demo resource.
Demo resource closed.
[AUDIT] BankService.transfer
[AUDIT] BankService.openAccount
[AUDIT] BankService.deposit
[AUDIT] BankService.withdraw
```

Figure 4. Concurrent deposit result, withdrawal policy checks, resource closing and audit messages.

13. How to Run the Project

The supplied README specifies JDK 17 or newer. In IntelliJ IDEA, open the extracted MiniBank_Project folder as a Maven project, allow the IDE to import pom.xml, and configure a JDK 17+ project SDK. Then open src/main/java/service/MiniBank.java and run its main method. Run the application with the project root as the working directory so the relative path data/accounts.txt resolves as intended.

Alternatively, from the project root in a terminal with JDK 17+, compile and run the source. The commands below follow the project README for PowerShell:

```
New-Item -ItemType Directory -Force out | Out-Null
```

```
javac -d out (Get-ChildItem -Recurse src/main/java -Filter *.java | ForEach-Object { $_.FullName
})
```

```
java -cp out service.MiniBank
```

The README also documents Maven commands: mvn test and mvn package. Maven is required for the configured JUnit test/build workflow and may need internet access on the first run to download dependencies.

14. Conclusion

MiniBank integrates core and intermediate Java object-oriented programming topics in a console banking simulation. It separates the user menu, service operations, account models, repository and utility/error-handling code. Its account types demonstrate inheritance and polymorphism, while the generic repository, custom exceptions, annotations, reflection, local persistence and concurrent transaction demonstration provide additional learning examples. The sample run shows account creation, deposits, transfers, statements, reporting and the concurrency demonstration. The application remains an educational local simulation and should not be used for real financial transactions.

15. References

- MiniBank_Project.zip supplied for this task: README.md, PRACTICAL_MAP.md, pom.xml and Java source files.



MiniBank_Project.zip

- GitHub link: https://github.com/D26CE149/CEUE203-OOP
- Java source package: src/main/java/ (service, model, repository, exception and util).
- JUnit test source: src/test/java/service/BankServiceTest.java.
- Project configuration and execution instructions: README.md and pom.xml.